

SIRCSS-CTA

Lab 4: Large Language Models

1 Few-shot classification with large language models

In-context learning is the idea to use large language models to complete tasks in a few-shot or zero-shot way. In this assignment, we will classify Swedish political manifesto quotes as belonging to either the Social Democratic party (**Socialdemokraterna**) or the Conservative party (**Moderaterna**).

The goal is to use an LLM as a classifier through an API from R, store the outputs in a structured format, and evaluate the classifier properly.

1.1 Install packages

Install the packages used in this assignment. The `uuml` package contains the course data and is installed from GitHub.

```
install.packages(c("ellmer", "caret", "remotes", "usethis"))
remotes::install_github("MansMeg/IntroML", subdir = "rpackage")
```

1.2 OpenAI API key

For this assignment, you need an OpenAI developer account and an API key [here](https://platform.openai.com/keys). A ChatGPT Plus subscription is not the same thing as API access. This will cost you approx \$6. Feel free to try other LLMs instead, through `ellmer`.

Warning: Never write your API key directly in your script, report, or any file that may be uploaded to a public repository. Store it as an environment variable instead.

Open your `.Renviron` file:

```
usethis::edit_r_environ()
```

Add the following line to the file, replacing the placeholder with your actual key:

```
OPENAI_API_KEY="[YOUR KEY GOES HERE]"
```

Save the file and restart R. You can then check that R can see the key:

```
Sys.getenv("OPENAI_API_KEY") != ""
```

1.3 First API call

We will use `gpt-5.4-mini` for development and the main classification task. The argument `echo = "none"` means that `ellmer` does not automatically print model output while it is generated. This is useful when we run many API calls. For a single test call, we can override this with `echo = "output"` so that the answer is printed.

```
library(ellmer)

chat_test <- chat_openai(
  model = "gpt-5.4-mini",
  system_prompt = "You are a helpful assistant.",
  params = params(temperature = 0),
  echo = "none"
)

chat_test$chat(
  "Who was Gustav Vasa? Answer in one sentence.",
  echo = "output"
)
```

Note! API calls cost money. Start with a very small subset to check that your prompt, schema, and evaluation code work before running the model on more observations.

1.4 Data

Note! This data is included in `uuml` version 0.4.0 and later.

```
library(uuml)
library(caret)

data("pc_test")
data("pc_train")

pc_train$party <- as.character(pc_train$party)
pc_test$party <- as.character(pc_test$party)

pc_train$party[pc_train$party == "S"] <- "Socialdemokraterna"
pc_train$party[pc_train$party == "M"] <- "Moderaterna"
pc_test$party[pc_test$party == "S"] <- "Socialdemokraterna"
pc_test$party[pc_test$party == "M"] <- "Moderaterna"

table(pc_train$party)
table(pc_test$party)
```

1.5 Structured outputs

Do not parse free text answers from the model. Instead, require a structured output with three fields: the predicted party, a confidence score, and a short reason.

```
party_labels <- c("Socialdemokraterna", "Moderaterna")

# This tells the model exactly what fields to return.
answer_format <- type_object(
  "Classification of one Swedish political manifesto quote.",
  party = type_enum(
    party_labels,
```

```

    "Predicted party."
  ),
  confidence = type_number("Confidence from 0 to 1."),
  reason = type_string("Very short reason for the classification.")
)

chat_mini <- chat_openai(
  model = "gpt-5.4-mini",
  system_prompt = "You classify Swedish political manifesto quotes.",
  params = params(temperature = 0),
  echo = "none"
)

```

1.6 Few-shot prompt

Create a small, balanced set of examples from the training data and use them to build a few-shot classification prompt.

```

set.seed(123)

s_examples <- pc_train[pc_train$party == "Socialdemokraterna", ]
m_examples <- pc_train[pc_train$party == "Moderaterna", ]

s_examples <- s_examples[sample(nrow(s_examples), 3), ]
m_examples <- m_examples[sample(nrow(m_examples), 3), ]

examples <- rbind(s_examples, m_examples)

examples_text <- paste0(
  "Quote: ", examples$quote,
  "\nParty: ", examples$party,
  collapse = "\n\n"
)

cat(examples_text)

```

1.7 Small pilot run

Before classifying many quotes, run the model on 10 test quotes. Inspect the returned `party`, `confidence`, and `reason` fields. If the output looks wrong, revise your prompt before continuing.

```

pc_pilot <- pc_test[sample(nrow(pc_test), 10), ]

pilot_prompts <- paste0(
  "Classify the Swedish political manifesto quote.\n\n",
  "Allowed labels: Socialdemokraterna or Moderaterna.\n\n",
  "Examples:\n\n", examples_text, "\n\n",
  "Quote to classify:\n\n", pc_pilot$quote
)

```

```

)

pilot_predictions <- parallel_chat_structured(
  chat_mini,
  pilot_prompts,
  type = answer_format,
  max_active = 2
)

names(pilot_predictions)[names(pilot_predictions) == "party"] <- "prediction"
pilot_results <- cbind(pc_pilot, pilot_predictions)
pilot_results[, c("quote", "party", "prediction", "confidence", "reason")]

```

1.8 Evaluate the mini model

After your pilot works, classify the full test set or a clearly justified subset of the test set. The `caret` package computes the confusion matrix and common classification metrics for us.

```

test_prompts <- paste0(
  "Classify the Swedish political manifesto quote.\n\n",
  "Allowed labels: Socialdemokraterna or Moderaterna.\n\n",
  "Examples:\n\n", examples_text, "\n\n",
  "Quote to classify:\n\n", pc_test$quote
)

mini_predictions <- parallel_chat_structured(
  chat_mini,
  test_prompts,
  type = answer_format,
  max_active = 2
)

names(mini_predictions)[names(mini_predictions) == "party"] <- "prediction"
mini_results <- cbind(pc_test, mini_predictions)

confusionMatrix(
  data = factor(mini_results$prediction, levels = party_labels),
  reference = factor(mini_results$party, levels = party_labels),
  mode = "everything"
)

```

1.9 Compare with GPT-5.5

Use the same prompt and the same evaluation cases to compare `gpt-5.4-mini` with `gpt-5.5`. If API costs are a concern, compare both models on a random subset of at least 30 test quotes and state clearly that you used a subset.

```

chat_55 <- chat_openai(
  model = "gpt-5.5",
  system_prompt = "You classify Swedish political manifesto quotes.",
  params = params(temperature = 0),
  echo = "none"
)

pc_compare <- pc_test[sample(nrow(pc_test), 30), ]

comparison_prompts <- paste0(
  "Classify the Swedish political manifesto quote.\n\n",
  "Allowed labels: Socialdemokraterna or Moderaterna.\n\n",
  "Examples:\n\n", examples_text, "\n\n",
  "Quote to classify:\n\n", pc_compare$quote
)

mini_compare <- parallel_chat_structured(
  chat_mini,
  comparison_prompts,
  type = answer_format,
  max_active = 2
)

gpt55_compare <- parallel_chat_structured(
  chat_55,
  comparison_prompts,
  type = answer_format,
  max_active = 2
)

names(mini_compare)[names(mini_compare) == "party"] <- "prediction"
names(gpt55_compare)[names(gpt55_compare) == "party"] <- "prediction"

confusionMatrix(
  data = factor(mini_compare$prediction, levels = party_labels),
  reference = factor(pc_compare$party, levels = party_labels),
  mode = "everything"
)

confusionMatrix(
  data = factor(gpt55_compare$prediction, levels = party_labels),
  reference = factor(pc_compare$party, levels = party_labels),
  mode = "everything"
)

```

Discuss whether `gpt-5.5` improves the classification enough to justify using the larger model for this task. Compare performance, confidence, errors, and practical issues such as cost.

1.10 What to hand in

Your report should include:

1. A short description of your few-shot prompt and how you selected the demonstrations.
2. The structured output schema used for the model response.
3. Results from the small pilot run, including any prompt changes you made after inspecting it.
4. Proper evaluation of your final `gpt-5.4-mini` classifier using `confusionMatrix()`.
5. A comparison between `gpt-5.4-mini` and `gpt-5.5` on the same evaluation cases.
6. A short discussion of what kinds of quotes the model misclassified and whether the larger model was worth using.